

Handout: 725103 Data Structure and Algorithm

สัปดาห์ที่ 7: Dynamic Arrays, Linked List, Stack และ Queue

ภาคปลาย ปีการศึกษา 2568 | อาจารย์ผู้สอน: ปพนธีร์ แยมโชติ

เป้าหมายการเรียนรู้ประจำสัปดาห์ (Learning Outcomes)

เมื่อจบคาบนี้ นักศึกษาสามารถ...

- อธิบายแนวคิดของ **dynamic array** (capacity, resize, copy) และทำ **trace** การขยายขนาดได้
- วาด/trace โครงสร้าง **singly linked list** และอธิบายการ **insert/delete** โดยอัปเดต **pointer** ได้ถูกต้อง
- อธิบาย **Stack (LIFO)** และ **Queue (FIFO)** ในมุมมอง ADT พร้อมทำ **trace** ลำดับคำสั่งได้
- เปรียบเทียบ **trade-off** ของ array vs linked list vs stack/queue ระดับแนวคิด (รวมถึง $O(1)$, $O(n)$)

คำศัพท์สำคัญ (Key Terms)

- dynamic array, capacity, resize, copy, amortized time
- linked list, node, pointer, next, head, tail, null
- Stack, push, pop, top, LIFO
- Queue, enqueue, dequeue, front, rear, FIFO
- $O(1)$, $O(n)$ (ระดับแนวคิด)

1 Dynamic Arrays — array ที่ “โตได้”

Definition 1 (Dynamic Array). **dynamic array** คือโครงสร้างข้อมูลที่มี “หน้าตาเหมือน array” (เข้าถึงด้วย index) แต่สามารถ **ขยายขนาด** ได้เมื่อเต็ม โดยมักใช้แนวคิด **capacity** (ความจุ) และ **resize** (ย้ายไปพื้นที่ใหม่ที่ใหญ่กว่าแล้ว copy ข้อมูลเดิม)

1.1 ปัญหาของ static array

Pain Point ของ array แบบกำหนดขนาดตายตัว

ถ้ากำหนดขนาดไว้ n ช่อง แล้วข้อมูลเพิ่มเกิน n :

- เก็บเพิ่มไม่ได้ (เต็ม)
- ทางเลือกคือสร้าง array ใหม่ที่ใหญ่กว่า แล้ว **copy** ของเดิมไปทั้งหมด

1.2 Visualization: capacity + size

Visualization: size vs capacity

ให้ size = จำนวนข้อมูลจริง, capacity = จำนวนช่องที่มี

ตัวอย่าง: size=6, capacity=8

0	1	2	3	4	5	6	7
12	7	25	9	16	4		

ช่อง 6--7 ยังว่างอยู่ (เผื่อโต)

1.3 กฎ resize ที่พบบ่อย: ``double capacity''

Idea: โตแบบคูณ 2 เพื่อให้เฉลี่ยเร็ว

เมื่อ size = capacity แล้ว append เข้ามาใหม่:

- สร้าง array ใหม่ที่มี capacity' = $2 \times \text{capacity}$
- copy ข้อมูลเดิมทั้งหมด size ขึ้น
- ใส่ข้อมูลใหม่ต่อท้าย

ข้อสังเกต: บางครั้ง append แพงมาก (ต้อง copy หลายขึ้น) แต่โดยเฉลี่ยมักมองเป็น **amortized $O(1)$** (ระดับ-แนวคิด)

1.4 Pseudocode: Append แบบ dynamic array (เชิงแนวคิด)

Algorithm 1 Append (Dynamic Array แบบมี resize)

Require: dynamic array A ที่มี size, capacity; ค่าใหม่ x

- 1: **if** size = capacity **then**
 - 2: สร้าง array ใหม่ B ที่มี capacity = $2 \times \text{capacity}$
 - 3: copy สมาชิกทั้งหมดจาก A ไป B
 - 4: ให้ $A \leftarrow B$
 - 5: **end if**
 - 6: ใส่ x ที่ $A[\text{size}]$
 - 7: size $\leftarrow \text{size} + 1$
-

1.5 Worksheet: Trace การ append และการ resize

Worksheet: Trace Table (เติม size/capacity และจำนวนที่ copy)

เริ่มต้น capacity = 4, size = 0 แล้วทำ append ค่า 1,2,3,4,5,6,7,8,9 ตามลำดับ (โตแบบคูณ 2)

append	size ก่อน	capacity ก่อน	resize? (Y/N)	#copy
1				
2				
3				
4				
5				
6				
7				
8				
9				

คำถามชวนคิด: แถวไหนที่แพงที่สุด? เพราะอะไร?

1.6 Operation + Complexity (ระดับแนวคิด)

Dynamic Array: Operation + Complexity (แนวคิด)

Operation	คำอธิบาย	Time
Access $A[i]$	เข้าถึงด้วย index	$O(1)$
Append (ทั่วไป)	ใส่ท้ายได้ทันทีถ้ายังไม่เต็ม	$O(1)$
Append (ตอน resize)	ต้อง copy ทั้งหมดก่อน	$O(n)$
Append (เฉลี่ย)	โตแบบคูณ 2 $\Rightarrow$ โดยเฉลี่ยมองเป็น amortized	amortized $O(1)$

Common Pitfalls (Dynamic Array)

- สับสน size กับ capacity (เติมเมื่อ size==capacity)
- ลืมว่า resize ต้อง **copy** ของเดิมทั้งหมด (จึงมีเคสที่แพง)
- เข้าใจผิดว่า append จะ $O(1)$ เสมอ (จริง ๆ มี worst-case $O(n)$)

2 Linked List — เก็บข้อมูลแบบ “โหนดต่อโหนด”

Definition 2 (Singly Linked List). **singly linked list** คือโครงสร้างข้อมูลที่เก็บเป็น **node** หลายตัว โดยแต่ละ node มี **data** และ pointer **next** ชี้ไป node ถัดไป (ตัวสุดท้ายชี้ **null**)

2.1 Visualization: node + next

Visualization: Singly Linked List

ตัวอย่าง list: head ชี้ไป node แรก

12 → 7 → 25 → 9 → null

หมายเหตุ: node ไม่จำเป็นต้องอยู่ติดกันใน memory (ไม่ contiguous) แต่เชื่อมกันด้วย pointer

2.2 Array vs Linked List (คิดแบบ operation)

Trade-off: ทำไมต้องมี Linked List?

- array: เข้าถึงด้วย index เร็ว ($O(1)$) แต่ insert/delete ตรงกลาง มักต้องเลื่อนหลายตัว
- linked list: insert/delete หลังตำแหน่งที่รู้แล้ว ทำได้เร็ว (อัปเดต pointer) แต่การไปให้ถึงตำแหน่งต้องเดินทีละ node

2.3 แก่นของคาบ: การอัปเดต pointer

กฎทอง: อย่า “ทำซ้ำซาก”

เวลาจะ insert/delete:

- ต้องอัปเดต pointer ให้ครบ และ ระวังลำดับ การชี้ (เก็บตัวชี้เดิมไว้ก่อน)
- เคสขอบ: list ว่าง, แทรกที่หัว, ลบหัว, ลบท้าย

2.4 Pseudocode: Insert หลังโหนด p (เชิงแนวคิด)

Algorithm 2 InsertAfter (Singly Linked List)

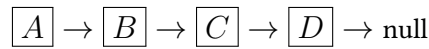
Require: โหนด p (มีอยู่ใน list), ค่าใหม่ x

- 1: สร้างโหนดใหม่ n ที่เก็บ x
 - 2: $n.next \leftarrow p.next$
 - 3: $p.next \leftarrow n$
-

2.5 Worksheet: Trace การ insert/delete ด้วยภาพ

Worksheet: เติมลูกศร (pointer) ให้ถูกต้อง

ให้ list เริ่มต้น:

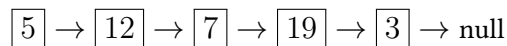


1. **InsertAfter(B, X):** แทรก $\boxed{X}$ หลัง $\boxed{B}$ แล้ววาดผลลัพธ์
2. **DeleteAfter(B):** ลบ node หลัง $\boxed{B}$ (จาก list ข้อ 1) แล้ววาดผลลัพธ์
3. **InsertHead(Y):** แทรก $\boxed{Y}$ ที่หัว list (ต้องอัปเดต head)

2.6 Trace Table: เดิน list เพื่อหา target

Worksheet: Trace การเดิน list (Traversal)

ให้ list:



ต้องการหา target = 19 โดยเริ่มจาก cur = head แล้วทำ cur = cur.next ทีละสเต็ป

step	cur.data	target	decision
1		19	
2		19	
3		19	
4		19	

2.7 Operation + Complexity (ระดับแนวคิด)

Linked List: Operation + Complexity (แนวคิด)

Operation	คำอธิบาย	Time
Search / Find	เดินทีละ node	$O(n)$
InsertAfter (รู้ตำแหน่งแล้ว)	อัปเดต pointer 2 เส้น	$O(1)$
DeleteAfter (รู้ตำแหน่งแล้ว)	ข้าม node ที่จะลบ	$O(1)$
Access by index	ต้องเดินไปเรื่อย ๆ	$O(n)$

3 Stack — “กองจาน” แบบ LIFO

Definition 3 (Stack (ADT)). Stack เป็น ADT ที่ทำงานแบบ LIFO (Last-In First-Out): ใส่ล่าสุด จะถูกนำออกก่อน

3.1 Operation ของ Stack

Stack Operations

- push(x): ใส่ข้อมูล x ที่ด้านบน (top)
- pop(): เอาข้อมูลบนสุดออก
- peek()/top(): ดูข้อมูลบนสุด (ไม่เอาออก)
- isEmpty(): ว่าว่างหรือไม่

3.2 Visualization

Visualization: LIFO

top
9
25
7
12

ถ้า push(4) จะวางบนสุด; ถ้า pop() จะเอา 9 ออก

3.3 Pseudocode: Push/Pop (เชิงแนวคิด)

Algorithm 3 Push และ Pop (Stack ADT)

Require: stack S , ค่า x

- 1: Push: วาง x ไว้ที่ top ของ S
 - 2: Pop: เอาค่าที่ top ของ S ออกและคืนค่านั้น
-

3.4 Worksheet: Trace คำสั่ง Stack

Worksheet: Trace Table (เติมสภาพ stack หลังแต่ละคำสั่ง)

เริ่มต้น stack ว่าง แล้วทำคำสั่ง:

push(3), push(5), pop(), push(2), push(8), pop()

ให้เติม สภาพของ stack (จากล่างขึ้นบน) หลังแต่ละ step

step	command	stack (bottom → top)
1	push(3)	
2	push(5)	
3	pop()	
4	push(2)	
5	push(8)	
6	pop()	

3.5 Mini-Application (trace): ตรวจวงเล็บด้วย Stack

Idea: Parentheses Matching (ไม่ต้องโค้ด เน้น trace)

กติกา (แนวคิด):

- เจอ (ให้ push
- เจอ) ให้ pop (ถ้าว่างอยู่ก่อน แปลว่าผิด)
- จบสตริงแล้ว stack ต้องว่างพอดี

Exercise 4 (Trace วงเล็บ). จงทำ trace ด้วย stack เพื่อตอบว่าแต่ละสตริง ถูก/ผิด

1. (())

2. ()()

3.6 Complexity

ทำไม Stack ถึงเร็ว?

โดยทั่วไป push/pop/peek ทำงานที่ด้านบนสุดเสมอ จึงมองเป็น $O(1)$ (ระดับแนวคิด)

4 Queue — “แถวต่อคิว” แบบ FIFO

Definition 5 (Queue (ADT)). Queue เป็น ADT ที่ทำงานแบบ FIFO (First-In First-Out): มาก่อน ออกก่อน

4.1 Operation ของ Queue

Queue Operations

- enqueue(x): เข้าคิวที่ท้าย (rear)
- dequeue(): ออกจากคิวที่หน้า (front)
- front()/peek(): ดูคนหน้าสุด (ไม่เอาออก)
- isEmpty(): ว่าว่างหรือไม่

4.2 Visualization

Visualization: FIFO

front $\boxed{A} \rightarrow \boxed{B} \rightarrow \boxed{C}$ rear

dequeue() จะเอา $\boxed{A}$ ออก; enqueue(D) จะต่อ $\boxed{D}$ ที่ท้าย

4.3 Worksheet: Trace คำสั่ง Queue

Worksheet: Trace Table (เติมสภาพคิวหลังแต่ละคำสั่ง)

เริ่มต้น queue ว่าง แล้วทำคำสั่ง:

enqueue(A), enqueue(B), enqueue(C), dequeue(), enqueue(D), dequeue()

ให้เติม สภาพของ queue (front $\rightarrow$ rear) หลังแต่ละ step

step	command	queue (front $\rightarrow$ rear)
1	enqueue(A)	
2	enqueue(B)	
3	enqueue(C)	
4	dequeue()	
5	enqueue(D)	
6	dequeue()	

4.4 Optional (Design Idea): Queue ด้วย array ต้องระวังอะไร?

Idea: ถ้าใช้ array ทำ queue

ถ้า enqueue ต่อท้าย และ dequeue เอาหน้าออก:

- ถ้าเลื่อนทุกตัวเวลา dequeue จะช้า ($O(n)$)
- จึงนิยมใช้แนวคิด **circular queue** โดยขยับ front/rear index แทนการเลื่อน

4.5 Complexity (ระดับแนวคิด)

Queue: เร็วเมื่อออกแบบให้ทำงานที่หัว/ท้าย

โดยทั่วไป enqueue/dequeue/peek ทำที่ปลายทั้งสอง จึงมองเป็น $O(1)$ (ระดับแนวคิด)

5 สรุปภาพรวม: เลือกใช้อะไรดี?

Cheat Sheet: โครงสร้าง $\leftrightarrow$ จุดเด่น

Structure	เหมาะกับสถานการณ์
Dynamic Array	ต้องการ access ด้วย index บ่อย ๆ, append บ่อย, ยอมรับเคส resize ที่บางครั้งแพงได้
Linked List	ต้อง insert/delete บ่อย (หลังตำแหน่งที่รู้แล้ว), ยอมรับว่าการหา/เดินอาจช้ากว่า
Stack (LIFO)	งานย้อนกลับ/ย้อนรอย, ตรวจสอบเล็บ, undo, call stack (แนวคิด)
Queue (FIFO)	งานเข้ามาตามลำดับ, ระบบคิว, งานประมวลผลตามลำดับเวลา

Checklist ตอนสรุป (1 นาที)

นักศึกษาควรตอบได้ว่า...

- dynamic array โตได้เพราะ **resize + copy** และ append มีทั้งเคสเร็วและเคสแพง
- linked list ต้องระวัง **pointer update** และเคสขอบ (head/empty/one node)
- stack คือ **LIFO** และ queue คือ **FIFO** และทำ trace ลำดับคำสั่งได้ถูก

6 การบ้านหลังเรียน (เน้น trace/diagram)

HW1: Address Calculation (ส่งเป็นรูป/ไฟล์ PDF ได้)

กำหนด array C เป็น `int` ($w = 4$ bytes), `base(C)=1200`

1. จงหา `addr(C[5])`

2. (Bonus) ถ้าเป็น 2D array ขนาด 3×4 (row-major), `addr(C[2][3])` คือเท่าไร?

HW2: Trace Binary Search 3 เคส (เจอ/ไม่เจอ/ค่าซ้ำ)

ให้ $A = [2, 5, 7, 9, 12, 15, 18, 21, 25, 30]$ (0-based)

(ก) เคสเจอ: $x = 18$ เติมตารางให้ครบจนพบ

step	low	high	mid	$A[mid]$	decision
1					
2					
3					

(ข) เคสไม่เจอ: $x = 8$ เติมตารางให้ครบจนจบรูป (สรุปว่า NotFound)

step	low	high	mid	$A[mid]$	decision
1					
2					
3					
4					

(ค) เคสค่าซ้ำ: ให้ $B = [1, 3, 3, 3, 5, 7, 9]$ และ $x = 3$

1. Binary Search แบบเดิม “มีโอกาส” คืน index ไหนบ้าง? (ตอบได้มากกว่า 1 ค่า)

2. ถ้าอยากได้ ตำแหน่งแรก ของ 3 ต้องทำอะไรเพิ่ม? (อธิบายแนวคิด ไม่ต้องเขียนโค้ด)

HW3: Dynamic Array Resize Trace (ส่งเป็นรูป/ไฟล์ PDF ได้)

เริ่มต้น dynamic array มี capacity=2, size=0 และโตแบบคูณ 2

1. ทำ trace การ append ค่า 10,20,30,40,50 โดยเติมตาราง (size/capacity/resize/#copy)

2. สรุปว่าเกิด resize กี่ครั้ง และรวม copy ทั้งหมดกี่ชิ้น

HW4: Linked List Pointer Update 4 เคส

ให้ list เริ่มต้น:

1 → 3 → 5 → 7 → null

จงวาดภาพผลลัพธ์พร้อมระบุว่า pointer ไหนถูกอัปเดตบ้าง

1. InsertHead(0)
2. InsertAfter(node ที่มีค่า 3, 4)
3. DeleteAfter(node ที่มีค่า 5)
4. DeleteHead()

HW5: Stack/Queue Trace (อย่างละ 1)

1. **Stack:** ทำ trace ของคำสั่ง push(A), push(B), push(C), pop(), push(D), pop()

2. **Queue:** ทำ trace ของคำสั่ง enqueue(1), enqueue(2), dequeue(), enqueue(3), enqueue(4), dequeue()
